



Fault Tolerance in Distributed Training

Building resilient systems for large-scale ML training

Why Fault Tolerance Matters

In distributed training environments:

- Operations span **hundreds to thousands of GPUs**
- Failures become **inevitable**, not just possible
- Without proper fault tolerance, entire jobs must restart from scratch
- This dramatically increases training costs and time-to-model



Robust infrastructure must recover gracefully

Types of Failures

Hardware

- GPU overheating
- VRAM corruption
- Power fluctuations
- Disk failures

Software

- Memory leaks
- CUDA driver bugs
- Framework deadlocks
- OOM exceptions

Network

- Node communication failures
- Intermittent packet loss
- Bandwidth saturation
- NIC failures

Scheduler/Cluster

- Kubernetes pod evictions
- Slurm job preemptions
- Resource contention
- Maintenance downtime

Each type requires specific strategies for graceful handling

Checkpointing

The foundation of fault tolerance in ML:

- Periodically save **model weights + optimizer state**
- Enables resuming training from last checkpoint
- Common formats: PyTorch `.pt`, TensorFlow `.ckpt`
- Frequency tradeoff: overhead vs. potential data loss

i **Industry standard** for LLMs, computer vision, and other compute-intensive training workloads



Checkpoint files can range from **MBs to TBs** for large models



Elastic Training



Dynamic Adaptation

Automatically adjusts to available GPU/node count



Resizable Jobs

Add/remove resources during active training runs

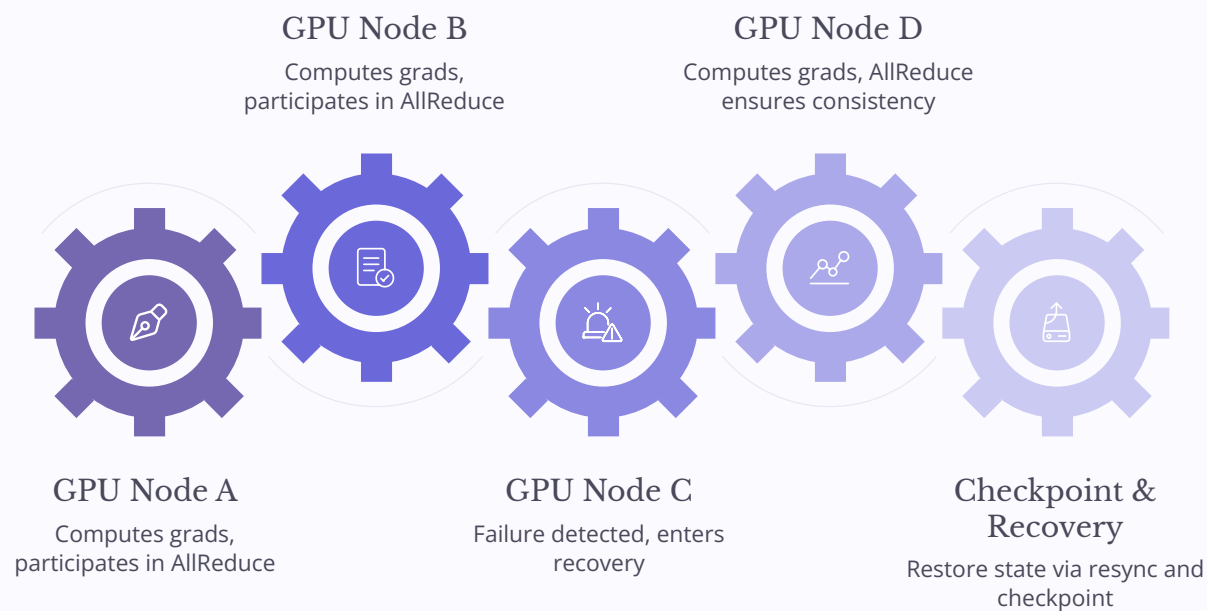


Failure Resilience

Prevents full restart when individual nodes fail

Technologies enabling elastic training: PyTorch Elastic, TorchElastic, Horovod Elastic, Ray Train, DeepSpeed Elastic

Gradient & State Synchronization



Critical Components

- Gradient consistency across nodes
- Optimizer state synchronization
- AllReduce-aware checkpointing

Complexity Factors

- Sharded optimizers (ZeRO, FSDP)
- Model & pipeline parallelism
- Resync speed at scale

Infrastructure-Level Fault Tolerance



Kubernetes

Pod auto-restarts +
rescheduling policies

Slurm

Job retries + checkpoint
continuation

Cluster Managers

Automatic task reassignment +
GPU health monitoring

Integration is key: Infrastructure systems must work in concert with ML framework checkpoint logic

Fault Tolerance in Practice



OpenAI GPT Training

Regular checkpointing every few hours, distributed across multiple data centers with redundant storage



Google DeepMind

Elastic training across thousands of TPUs with sophisticated checkpoint orchestration



Netflix

Auto-healing ML pipelines with Kubernetes and container-based recovery mechanisms



Financial Services

Compliance-critical AI systems with mandatory fault tolerance to meet regulatory requirements

Without proper fault tolerance: weeks of training can be lost overnight

Challenges in Fault Tolerance

- **Storage Bottlenecks**

Checkpoint files can reach GBs–TBs, creating I/O and storage challenges

- **Model Parallelism Complexity**

Elastic training becomes significantly harder with model parallelism strategies

- **Restart Overhead**

Poorly sharded jobs face high overhead during recovery operations

- **Performance-Safety Tradeoff**

Every fault tolerance mechanism introduces some performance impact



Still an active research and infrastructure engineering challenge



Best Practices



Automate Checkpointing

Use versioned cloud storage (S3/GCS) with consistent naming conventions



Incremental Checkpoints

Save only changed weights instead of full model dumps to reduce I/O



Test Recovery Flows

Validate failure recovery before production training runs



Multi-Level Resilience

Combine elastic training with cluster-level failover mechanisms

Always budget **extra cost and time** for resilience measures

Key Takeaways

100%

Failure Rate

At scale, failures are guaranteed, not merely possible

10x

Cost Impact

Training costs can increase by an order of magnitude without proper fault tolerance

Remember

- Infrastructure and ML frameworks must work together
- Fault tolerance isn't optional at scale
- Prevention costs less than recovery

Questions?

3

Key Strategies

Checkpointing, elastic training, and infrastructure resilience form the core defense